

Push Notifications

Backend integration guide

CBS Kitchenware mobile app (Android & iOS, built with Expo / React Native). This document describes everything the backend needs so the app can receive push notifications: storing device tokens, two new endpoints, sending through the Expo Push API, when to send, the admin panel, ready-to-adapt C# code and how to test it.

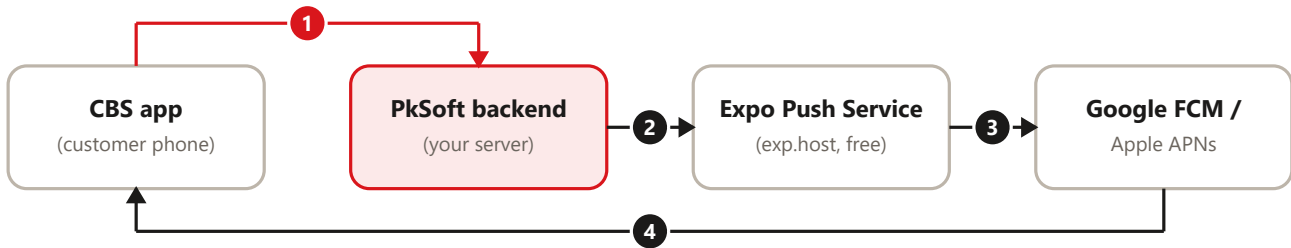
Item	Value
App	CBS Kitchenware (D2C) - Android package / iOS bundle id com.cbskitchenware.trade
API it talks to	https://api.pksoftabh.com (dev: https://dev-api.pksoftabh.com)
Push provider	Expo Push Service (free) on top of Google FCM and Apple APNs
Expo project	owner arshdeepecomwork , project id b759dd5c-e39f-4587-b373-e13783aecca6
Prepared by	Mobile app team

What we need from the backend - at a glance

#	Task	Section
1	A PushTokens table: which device token belongs to which customer	§3
2	Two endpoints: register a token, remove a token	§4
3	A small send service that calls the Expo Push API (one HTTP POST)	§5, §9
4	Automatic notifications when an order changes status	§7
5	An admin panel form to send offers / announcements	§8
6	Clean-up of dead tokens (uninstalled apps)	§5.6, §9

1 How push notifications work

Only Google (Android) and Apple (iPhone) can wake a phone up to show a notification. Expo provides a free service in front of both, so the backend makes **one simple HTTPS call** for every platform instead of integrating with Google and Apple separately.



- 1 After sign-in the app sends its **push token** to the backend (Register endpoint, §4).
- 2 When something happens, the backend **POSTs the message** to <https://exp.host/--/api/v2/push/send> (§5).
- 3 Expo **forwards** it to Google (Android) or Apple (iPhone).
- 4 The phone **shows the notification** - app open, in background or closed. A tap opens the right screen (§6).

Party	Role
CBS app (phone)	Asks the customer for permission, gets this phone's push token (its "address") from Expo, and sends it to the backend after sign-in. Shows notifications and opens the right screen when one is tapped.
PkSoft backend	Stores tokens per customer. Decides when to notify and what to say. Sends the message to Expo.
Expo Push Service	Receives the message at https://exp.host/--/api/v2/push/send and hands it to Google or Apple. No server to run.
Google FCM / Apple APNs	Deliver the notification to the phone - even if the app is closed.

1.1 The three moments

When	What happens	Who
Customer signs in	App gets its push token (e.g. <code>ExponentPushToken[xxxxxxxxxxxxxxxxxxxxxx]</code>) and calls Register token . Backend saves it against the customer.	App → Backend
Something happens (order shipped, offer...)	Backend looks up the customer's tokens and POSTs a message to Expo. Expo delivers it to the phone.	Backend → Expo → Phone
Customer taps it	The app reads the hidden data of the notification (e.g. <code>{"screen": "order", "id": "..."} </code>) and opens that screen.	App

1.2 Who does what

Task	Owner	Status
Permission prompt, notification channels, showing notifications, opening the right screen on tap	App team	Done
Getting the push token, calling Register on sign-in and Remove on sign-out	App team	Ready once §4 exists
Firebase (FCM) credentials uploaded to Expo, Apple credentials (iOS)	App team	In progress
PushTokens table + Register / Remove endpoints	Backend	To do (§3, §4)
Send service (Expo Push API), dead-token clean-up, receipts	Backend	To do (§5, §9)
Automatic notifications on order status changes	Backend	To do (§7)
Admin panel "send notification" form	Backend / admin	To do (§8)

GOOD TO KNOW

The backend does **not** need any Google or Apple keys. Those are configured once on the Expo side by the app team. The backend only talks to Expo over plain HTTPS. (An optional Expo access token can be added for extra security - see §5.7.)

2 What the app does (so you know what to expect)

- After sign-in the app asks permission ("Allow CBS Kitchenware to send you notifications?"). If the customer refuses, no token is sent - this is normal and the backend should simply not have a token for that customer.
- It gets an **Expo push token** - a string like `ExponentPushToken[aBc123XyZ456_-]` (some older SDKs print `ExpoPushToken[...]`; accept both).
- It calls **Register token** (§4.1) on every app start while signed in, and again if the token changes. Treat it as an **idempotent upsert** - repeat calls with the same token are expected.
- On **sign-out** it calls **Remove token** (§4.2) before clearing its session, so a shared phone stops receiving the previous customer's notifications.
- It has two Android notification channels: **orders** (high priority - "Order updates") and **offers** (normal - "Offers & new arrivals"). Customers can mute offers in phone settings and still get order updates.
- When a notification is tapped the app opens the screen described by its **data** (§6). Unknown or missing **data** opens the Home screen - it never crashes.

One customer can have **several tokens** (phone + tablet, or a reinstall). One token belongs to **one device**, and therefore to one customer at a time.

3 Database

3.1 PushTokens table (required)

SQL Server

```

CREATE TABLE PushTokens (
  Id          BIGINT IDENTITY(1,1) PRIMARY KEY,
  UserCode    UNIQUEIDENTIFIER NOT NULL,      -- customer code (userModel.code / "uid")
  Token       NVARCHAR(255)    NOT NULL,      -- ExponentPushToken[...]
  Platform    VARCHAR(10)      NOT NULL,      -- 'android' | 'ios'
  DeviceName  NVARCHAR(100)    NULL,          -- e.g. "Pixel 7", for support/debugging
  AppVersion  VARCHAR(20)      NULL,          -- e.g. "1.0.0"
  IsActive    BIT              NOT NULL DEFAULT 1,
  CreatedAt   DATETIME2        NOT NULL DEFAULT SYSUTCDATETIME(),
  LastSeenAt  DATETIME2        NOT NULL DEFAULT SYSUTCDATETIME(), -- updated on every Register call
  CONSTRAINT UQ_PushTokens_Token UNIQUE (Token) -- one row per device
);

CREATE INDEX IX_PushTokens_UserCode ON PushTokens (UserCode) WHERE IsActive = 1;

```

Rules

- **Token** is **unique**. If Register arrives with a token that already exists for a different customer, **move** it (update **UserCode**) - the phone now belongs to whoever is signed in.
- Never send to rows with **IsActive = 0**. Optionally delete inactive rows older than 90 days.
- Rows whose **LastSeenAt** is older than ~6 months can be deactivated: the app refreshes it on every launch.

3.2 NotificationLog table (recommended)

Keeps a history for support ("did we notify this customer?") and holds Expo ticket ids for the receipt check (\$5.6).

SQL Server

```

CREATE TABLE NotificationLog (
  Id          BIGINT IDENTITY(1,1) PRIMARY KEY,
  UserCode    UNIQUEIDENTIFIER NULL,
  Token       NVARCHAR(255)    NOT NULL,
  Channel     VARCHAR(20)      NOT NULL,      -- 'orders' | 'offers'
  Title       NVARCHAR(100)    NOT NULL,
  Body        NVARCHAR(300)    NOT NULL,
  DataJson    NVARCHAR(1000)   NULL,          -- the "data" object sent
  EventKey    VARCHAR(100)     NULL,          -- e.g. 'order:<orderCode>:shipped' (prevents duplicates)
  TicketId    VARCHAR(64)      NULL,          -- Expo ticket id when status = ok
  TicketStatus VARCHAR(10)     NULL,          -- 'ok' | 'error'
  ReceiptStatus VARCHAR(10)    NULL,          -- filled by the receipt job
  Error       VARCHAR(100)     NULL,          -- e.g. DeviceNotRegistered
  CreatedAt   DATETIME2        NOT NULL DEFAULT SYSUTCDATETIME(),
  ReceiptCheckedAt DATETIME2    NULL
);

CREATE INDEX IX_NotificationLog_Receipts ON NotificationLog (CreatedAt)
  WHERE TicketStatus = 'ok' AND ReceiptCheckedAt IS NULL;
CREATE INDEX IX_NotificationLog_EventKey ON NotificationLog (EventKey);

```

3.3 Notification preferences (optional, later)

If customers should be able to switch off offers inside the app (not only in phone settings), add **NotifyOrders BIT DEFAULT 1** and **NotifyOffers BIT DEFAULT 1** to the customer table and check them before sending. Not needed for the first release.

4 New endpoints for the app

Please follow the same conventions as the existing customer endpoints (e.g. `api/pk/Customer/Cart/add`):

Convention	Same as today
Auth headers	<code>Authorization: Bearer <JWT></code> , <code>X-AUTH: <pksoft_token></code> (from <code>api/pk/Customer/auth</code>)
Origin	Whitelisted <code>Origin</code> header (the app sends the configured origin)
CSRF	<code>X-CSRF-TOKEN</code> from <code>GET api/pk/Customer/verify-cs</code> on POST requests
Timestamp	<code>d</code> = base64 of the client date string, e.g. <code>base64("Tue Oct 06 2026 16:56:15 GMT+0530 (India Standard Time)")</code>
Response	Envelope <code>{ status, messageCode, message, data }</code> - <code>messageCode 100</code> = success

NOTE

The paths below are **suggestions**. Use whatever fits your routing - just tell the app team the final paths and message codes.

4.1 Register token

Request

POST `api/pk/Customer/PushToken/Register`

```
{
  "uid":          "c764d774-d6d8-44c2-843a-b5e588e6a122",    // signed-in customer code
  "token":        "ExponentPushToken[aBc123XyZ456_-]",
  "platform":     "android",                                // "android" | "ios"
  "deviceName":   "Pixel 7",                                // optional
  "appVersion":   "1.0.0",                                   // optional
  "d":            "VHV1IE9jdCAwNiAyMDI2IDE2OjU2OjE1IEdNVCswNTMwIChJbmRpYSBTdGFuZGFyZCZBUaW1lKQ=="
}
```

Response - success

```
{ "status": 200, "messageCode": 100, "message": "Token registered.", "data": null }
```

Behaviour

1. Check `uid` is the customer of the JWT (reject otherwise, like other customer endpoints).
2. Validate the token format: `^Expo(nent)?PushToken\[ [A-Za-z0-9_\- ]+\]$`. Validate `platform` is android or ios.
3. **Upsert by token**: if the token exists, set `UserCode = uid`, `IsActive = 1`, `LastSeenAt = now`, and update platform / deviceName / appVersion. Otherwise insert a new row.
4. Return `100`. Calling it again with the same data must also return `100` (idempotent).

messageCode	When
100	Saved (new or updated)
102	Missing / invalid fields ("Please fill all mandatory Inputs." - as today)
401 (HTTP)	Invalid or expired JWT - as today

4.2 Remove token

Request
<pre>POST api/pk/Customer/PushToken/Remove { "uid": "c764d774-d6d8-44c2-843a-b5e588e6a122", "token": "ExponentPushToken[aBc123XyZ456_-]", "d": "VHV1IE9jdCAwNi..." }</pre>
Response
<pre>{ "status": 200, "messageCode": 100, "message": "Token removed.", "data": null }</pre>

- Deactivate (`IsActive = 0`) or delete the row where `Token = token AND UserCode = uid`.
- Return **100 even if the token was not found** - sign-out must never fail because of this.
- If the JWT has already expired the app cannot call this. That is fine: the next customer who signs in on that phone takes the token over (§3.1), and dead tokens are cleaned up by §5.6.

5 Sending: the Expo Push API

5.1 Endpoint

Item	Value
URL	<code>https://exp.host/--/api/v2/push/send</code>
Method	<code>POST</code>
Headers	<code>Content-Type: application/json</code> <code>Accept: application/json</code> <code>Accept-Encoding: gzip, deflate</code> <code>Authorization: Bearer <EXPO_ACCESS_TOKEN></code> - only if push security is enabled (§5.7)
Body	One message object, or an array of up to 100 message objects

GOOD TO KNOW

About the URL: `exp.host` is Expo's server and `/--/` is simply a fixed part of Expo's address (a separator Expo uses in its URLs). Copy the URL exactly - nothing in it needs to be filled in.

5.2 Message fields

Field	Type	Description
<code>to</code>	<code>string string[]</code>	Required. The Expo push token(s) from the PushTokens table. This is how the backend "addresses" the phone.
<code>title</code>	<code>string</code>	Bold first line. Keep it short (≈ 40-50 characters).
<code>body</code>	<code>string</code>	Message text (≈ 120-150 characters show fully on most phones).
<code>data</code>	<code>object</code>	Hidden JSON the app uses to open the right screen - see §6. Must be an object.
<code>channelId</code>	<code>string</code>	Android channel: <code>"orders"</code> or <code>"offers"</code> . Always send it.
<code>sound</code>	<code>string</code>	<code>"default"</code> to play the normal notification sound.
<code>priority</code>	<code>string</code>	<code>"default"</code> <code>"normal"</code> <code>"high"</code> . Use <code>"high"</code> for order updates.

Field	Type	Description
ttl	number	Seconds the message may wait for delivery if the phone is offline (e.g. 86400 for offers).
badge	number	iOS app-icon badge number (optional).
subtitle	string	iOS only, optional.

5.3 Examples

One customer, one device

POST https://exp.host/--/api/v2/push/send
Content-Type: application/json

```
{
  "to": "ExponentPushToken[aBc123XyZ456_-]",
  "title": "Your order has shipped",
  "body": "Order #Pk/26-27/00020 is on its way. Tap to track it.",
  "data": { "screen": "order", "id": "d519352d-6e8a-4434-b62e-dd6e27e8b8f4" },
  "channelId": "orders",
  "sound": "default",
  "priority": "high"
}
```

One customer with two devices - "to" can be a list

```
{ "to": ["ExponentPushToken[aaa...]", "ExponentPushToken[bbb...]"], "title": "...", "body": "...",
  "data": { "screen": "order", "id": "..." }, "channelId": "orders", "sound": "default" }
```

Different messages in one request - an array (max 100)

```
[
  { "to": "ExponentPushToken[aaa...]", "title": "Your order has shipped", "body": "...",
    "data": { "screen": "order", "id": "..." }, "channelId": "orders" },
  { "to": "ExponentPushToken[ccc...]", "title": "Delivered!", "body": "...",
    "data": { "screen": "order", "id": "..." }, "channelId": "orders" }
]
```

5.4 Response: push tickets

Expo answers immediately with **one ticket per message, in the same order** as sent:

```
{
  "data": [
    { "status": "ok", "id": "XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX" },
    { "status": "error",
      "message": "\"ExponentPushToken[ccc...]\" is not a registered push notification recipient",
      "details": { "error": "DeviceNotRegistered" } }
  ]
}
```

NOTE

"status": "ok" means Expo **accepted** the message - not that it reached the phone. Final delivery is confirmed by receipts (§5.6).

5.5 Limits and errors

Rule	Value / action
Messages per request	Max 100 - split larger sends into batches of 100
Rate limit	600 notifications per second per project
Payload size	Total message at most 4 KiB (keep data small)
HTTP 429 or 5xx	Retry with exponential backoff (e.g. 1s, 2s, 4s, 8s), then give up and log
Gzip	Request bodies may optionally be gzip-compressed

Error (in ticket or receipt)	Meaning	What the backend should do
DeviceNotRegistered	App uninstalled / token no longer valid	Deactivate that token and stop sending to it
MessageTooBig	Payload over 4 KiB	Shorten title/body/data; log it
MessageRateExceeded	Sending too fast to one device	Slow down / back off
MismatchSenderId	FCM credentials mismatch	Tell the app team (configuration issue on the Expo side)
InvalidCredentials	Push credentials invalid / expired	Tell the app team (configuration issue on the Expo side)
TOO_MANY_REQUESTS (request error)	Over 600/s	Back off and retry

5.6 Receipts (delivery confirmation)

A background job confirms delivery and catches tokens that died after sending:

```
POST https://exp.host/--/api/v2/push/getReceipts
Content-Type: application/json

{ "ids": ["XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX", "YYYYYYYY-YYYY-YYYY-YYYY-YYYYYYYYYYYY"] }

-> { "data": {
  "XXXXXXXX-...": { "status": "ok" },
  "YYYYYYYY-...": { "status": "error", "message": "...",
    "details": { "error": "DeviceNotRegistered" } }
} }
```

- Run every ~15 minutes. Expo recommends checking receipts **about 15 minutes after sending**; receipts are **cleared after 24 hours**.
- Up to **1000 ids** per request.
- On **DeviceNotRegistered** deactivate the token. On **MismatchSenderId** / **InvalidCredentials** alert the team.

5.7 Security (optional but recommended)

- By default anyone who knows a token could send to it. Tokens are long and only stored by the backend, so risk is low.
- To allow **only our backend** to send: the app team enables **push security** in the Expo (EAS) dashboard and creates an **access token**. The backend adds **Authorization: Bearer <EXPO_ACCESS_TOKEN>** to every Expo request.
- Keep that access token in server configuration (e.g. **appsettings.json** / environment variable / key vault), never in the app and never in source control.

6 The "data" contract (what the app opens)

Every notification should carry a **data** object telling the app which screen to open when it is tapped. These are the values the app understands today:

data	Opens	Where the id comes from
{ "screen": "order", "id": "<orderCode>" }	That order's detail / tracking	orderCode (GUID) from User/Orders
{ "screen": "orders" }	Orders list	-
{ "screen": "product", "id": "<mappingCode>" }	Product page	mappingCode (seller-price GUID), as used by Cart/add
{ "screen": "category", "id": "<code>", "name": "<name>" }	Category product list	Category code and name from load/category (name is required - listings are queried by name)
{ "screen": "categories" }	All categories	-
{ "screen": "cart" }	Cart	-
{ "screen": "wishlist" }	Wishlist	-
{ "screen": "home" }	Home	-

- Anything else (or no **data**) opens Home. New screens can be added later - agree the name with the app team first.
- Keys and screen names are **lower-case** and case-sensitive.
- Do not put personal data (addresses, phone numbers, payment info) in **data**, **title** or **body**.

6.1 Channels

channelId	Android name	Use for	Priority
orders	Order updates	Anything about the customer's own orders and payments	high
offers	Offers & new arrivals	Promotions, sales, new products, reminders	default

7 When to send: automatic notifications

Send **one notification per order** (not per product line - **User/Orders** returns one row per line, but the customer should get one message per order) whenever its status changes:

Event	Title	Body	data / channel
Order placed (after Order/success)	Order placed	Thanks! Order #{orderNo} for ₹{total} is confirmed.	order, orders
Confirmed by seller	Order confirmed	Order #{orderNo} has been confirmed and is being prepared.	order, orders
Shipped / dispatched	Your order has shipped	Order #{orderNo} is on its way. Tap to track it.	order, orders
Out for delivery	Out for delivery today	Order #{orderNo} will reach you today.	order, orders

Event	Title	Body	data / channel
Delivered	Delivered	Order #{orderNo} was delivered. Enjoy your purchase!	order, orders
Cancelled	Order cancelled	Order #{orderNo} was cancelled. Any amount paid will be refunded.	order, orders
Online payment failed	Payment failed	Your payment for order #{orderNo} did not go through. Tap to try again.	order, orders
Refund processed	Refund processed	₹{amount} for order #{orderNo} is on its way to you.	order, orders

Wording is a suggestion - emojis are allowed in title and body (e.g. a truck for "shipped"). Use the same status texts the orders API already returns ("New Order", "Processing", "Shipped", "Delivered", "Cancelled", ...).

Implementation rules

1. Send **after** the status change is committed to the database - never inside the transaction.
2. Send **asynchronously** (background queue / hosted service / Hangfire) so order APIs and the admin panel are not slowed down or broken if Expo is slow.
3. **No duplicates:** use an event key such as `order:{orderCode}:shipped` in NotificationLog and skip if it was already sent.
4. Send to **all active tokens** of the order's customer (`to` can be a list).
5. A failed notification must **never** fail or roll back the order update - log it and move on.
6. Customers without tokens (web shoppers, permission refused) simply get nothing - not an error.

8 Admin panel: manual notifications

A simple form for sales, new arrivals and announcements:

Field	Details
Title	Required, max 50 characters
Message	Required, max 150 characters
Audience	(a) All customers with a token (b) One customer - search by email / mobile (c) optional later: customers who ordered in the last N days
Opens	Home / Category (pick one) / Product (pick one) / Cart / Wishlist / Orders - builds the <code>data</code> object (§6)
Channel	<code>offers</code> by default (<code>orders</code> only for order-related messages)
Send test	Sends to the admin's own device token first, to check wording and the tap target
Send	Confirmation dialog showing how many devices will receive it
Schedule	Optional later: send at a chosen date/time

Sending to everyone: read active tokens in pages, send in batches of 100 (respecting 600/s), and show a result summary - sent, failed, and tokens removed (`DeviceNotRegistered`). Keep a history list of past sends (from NotificationLog) with date, title, audience and counts.

9 Reference implementation (C# / .NET 6+)

Ready to adapt - only `HttpClient` and `System.Text.Json` are used. Alternatively there is a community .NET library listed in Expo's docs ([expo-server-sdk-dotnet](#)).

Models

```
using System.Net.Http.Headers;
using System.Net.Http.Json;
using System.Text.Json;
using System.Text.Json.Serialization;
using Microsoft.Extensions.Configuration; // IConfiguration (implicit in ASP.NET Core)

public sealed record ExpoPushMessage(
    string To,
    string Title,
    string Body,
    object? Data = null,
    string ChannelId = "orders",
    string Sound = "default",
    string Priority = "high",
    int? Ttl = null);

public sealed class ExpoSendResponse { public List<ExpoTicket>? Data { get; set; } }

public sealed class ExpoTicket
{
    public string Status { get; set; } = ""; // "ok" | "error"
    public string? Id { get; set; } // ticket id, for receipts
    public string? Message { get; set; }
    public ExpoErrorDetails? Details { get; set; }
}

public sealed class ExpoErrorDetails { public string? Error { get; set; } } // e.g. DeviceNotRegistered
```

Send service

```
public sealed class ExpoPushService
{
    private const string SendUrl = "https://exp.host/--/api/v2/push/send";
    private static readonly JsonSerializerOptions Json = new(JsonSerializerDefaults.Web) // camelCase
    {
        DefaultIgnoreCondition = JsonIgnoreCondition.WhenWritingNull
    };

    private readonly HttpClient _http;
    private readonly IPushTokenRepository _tokens; // your PushTokens / NotificationLog access
    private readonly string? _accessToken; // optional - only if push security is enabled

    public ExpoPushService(HttpClient http, IPushTokenRepository tokens, IConfiguration config)
    {
        _http = http;
        _tokens = tokens;
        _accessToken = config["Expo:AccessToken"];
    }

    /// Sends one notification to every active device of a customer.
    public async Task SendToCustomerAsync(Guid userCode, string title, string body, object data,
        string channelId, string? eventKey = null,
        CancellationToken ct = default)
    {
        if (eventKey != null && await _tokens.WasSentAsync(eventKey, ct)) return; // no duplicates
        var tokens = await _tokens.GetActiveTokensAsync(userCode, ct);
        if (tokens.Count == 0) return; // nothing to do
        var messages = tokens.Select(t => new ExpoPushMessage(t, title, body, data, channelId)).ToList();
        await SendAsync(messages, userCode, eventKey, ct);
    }
}
```

Send service

```

/// Sends any number of messages in batches of 100.
public async Task SendAsync(IReadOnlyList<ExpoPushMessage> messages, Guid? userCode = null,
    string? eventKey = null, CancellationToken ct = default)
{
    foreach (var batch in messages.Chunk(100))
    {
        var tickets = await PostWithRetryAsync(batch, ct);
        for (var i = 0; i < batch.Length; i++)
        {
            var ticket = i < tickets.Count ? tickets[i] : null;
            await _tokens.LogAsync(batch[i], userCode, eventKey, ticket, ct); // log row
            if (ticket?.Details?.Error == "DeviceNotRegistered")
                await _tokens.DeactivateAsync(batch[i].To, ct);
        }
    }
}

private async Task<List<ExpoTicket>> PostWithRetryAsync(ExpoPushMessage[] batch, CancellationToken ct)
{
    for (var attempt = 0; ; attempt++)
    {
        using var request = new HttpRequestMessage(HttpMethod.Post, SendUrl)
        {
            Content = JsonContent.Create(batch, options: Json)
        };
        request.Headers.Accept.ParseAdd("application/json");
        if (!string.IsNullOrEmpty(_accessToken))
            request.Headers.Authorization = new AuthenticationHeaderValue("Bearer", _accessToken);

        using var response = await _http.SendAsync(request, ct);
        var code = (int)response.StatusCode;
        if ((code == 429 || code >= 500) && attempt < 4)
        {
            await Task.Delay(TimeSpan.FromSeconds(Math.Pow(2, attempt)), ct); // 1s, 2s, 4s, 8s
            continue;
        }
        response.EnsureSuccessStatusCode();
        var result = await response.Content.ReadFromJsonAsync<ExpoSendResponse>(Json, ct);
        return result?.Data ?? new List<ExpoTicket>();
    }
}

```

Registration

```

// Program.cs
builder.Services.AddHttpClient<ExpoPushService>(c => c.Timeout = TimeSpan.FromSeconds(30));

```

Using it when an order ships

```

// after the new status is saved and committed - ideally enqueued to a background worker
await _push.SendToCustomerAsync(
    order.UserCode,
    title: "Your order has shipped",
    body: $"Order #{order.OrderNo} is on its way. Tap to track it.",
    data: new { screen = "order", id = order.OrderCode },
    channelId: "orders",
    eventKey: $"order:{order.OrderCode}:shipped");

```

Receipt check job

```
// Runs every 15 minutes (hosted service / Hangfire / scheduled job)
var pending = await _tokens.GetTicketsToCheckAsync(olderThan: TimeSpan.FromMinutes(15), max: 1000, ct);
if (pending.Count > 0)
{
    // if push security is on, add the same Authorization: Bearer <EXPO_ACCESS_TOKEN> header here
    var response = await _http.PostAsJsonAsync("https://exp.host/--api/v2/push/getReceipts",
        new { ids = pending.Select(p => p.TicketId) }, ct);
    var receipts = await response.Content.ReadFromJsonAsync<
        Dictionary<string, Dictionary<string, ExpoTicket>>>(cancellationToken: ct);
    foreach (var (ticketId, receipt) in receipts?.GetValueOrDefault("data") ?? new())
    {
        await _tokens.SaveReceiptAsync(ticketId, receipt.Status, receipt.Details?.Error, ct);
        if (receipt.Details?.Error == "DeviceNotRegistered")
            await _tokens.DeactivateByTicketAsync(ticketId, ct);
        // MismatchSenderId / InvalidCredentials -> alert the team (credentials problem)
    }
}
```

10 Testing

10.1 Before the endpoints exist

1. The app team installs the development build on a test phone; the app prints its push token in the logs and shares it with you.
2. Send a test from any PC - you should see the notification on the phone within seconds:

curl (macOS / Linux / Git Bash)

```
curl -X POST https://exp.host/--api/v2/push/send \
-H "Content-Type: application/json" \
-d '{"to": "ExponentPushToken[PASTE_TOKEN]", "title": "Test from backend",
  "body": "Hello from PkSoft", "data": {"screen": "orders"}, "channelId": "orders"}'
```

PowerShell (Windows)

```
$body = @'
{ "to": "ExponentPushToken[PASTE_TOKEN]", "title": "Test from backend",
  "body": "Hello from PkSoft", "data": { "screen": "orders" }, "channelId": "orders" }
'@

Invoke-RestMethod -Method Post -Uri "https://exp.host/--api/v2/push/send" `
-ContentType "application/json" -Body $body
```

Or use Postman with the same URL and JSON body, or Expo's web tool at <https://expo.dev/notifications>.

10.2 Test cases

#	Test	Expected
1	Sign in on the app	Register endpoint called; row in PushTokens with IsActive = 1
2	Restart the app	Register called again; same row, LastSeenAt updated (no duplicate row)
3	Sign in as customer B on the same phone	Same token row now has customer B's UserCode
4	Sign out	Remove called; row inactive; no more notifications to that phone

#	Test	Expected
5	Change an order to Shipped	Exactly one notification, channel "orders"; tapping opens that order
6	Change the same status again	No second notification (event key)
7	Customer with 2 devices	Both devices receive it
8	Uninstall the app, then send	Ticket or receipt returns DeviceNotRegistered; token deactivated
9	Admin "send test", then "send to all"	Test reaches admin device; summary shows sent/failed counts
10	Offer with Category target	Tapping opens that category's products
11	Expo unreachable (simulate)	Order update still succeeds; error logged; retries with backoff

11 Questions for the backend team

1. Final paths and message codes for **Register** and **Remove** token (we will plug them into the app).
2. The complete list of order **status texts** the orders API can return, and which events should notify the customer.
3. Where in the backend order statuses change (admin panel, seller panel, courier webhook?) - every place should trigger \$7.
4. Background job framework you prefer (hosted service, Hangfire, ...) for sending and receipts.
5. Should we enable Expo **push security** (\$5.7)? If yes, the app team will create the access token and hand it over securely.
6. Timeline - the app side is ready to connect as soon as the Register endpoint is available on dev-api.pksoftabh.com.

GOOD TO KNOW

Contact the mobile app team for the test-device token, the Expo dashboard, or any change to the **data** contract.

A Glossary

Term	Meaning
Push token	The address of one app install on one device, e.g. <code>ExponentPushToken[...]</code> . Not secret, but only store it for signed-in customers.
Expo Push Service	Expo's free service at exp.host that forwards messages to Google and Apple.
FCM / APNs	Firebase Cloud Messaging (Google, Android) and Apple Push Notification service (iOS).
Channel	Android category of notifications (here: orders, offers); customers can mute each one separately.
Ticket	Expo's immediate answer per message: accepted (ok) or rejected (error).
Receipt	Expo's later answer per ticket: delivered to Google/Apple or failed. Check after ~15 min, kept 24 h.
DeviceNotRegistered	The token is dead (app uninstalled / reset) - stop sending to it.
data	Hidden JSON in a notification that tells the app which screen to open.